

DataSource in Terraform

To Begin with the Lab

Summary of the Lab

In this lab, we learn how to use Terraform Data Sources to fetch information dynamically from AWS instead of hardcoding values in Terraform files. Rather than manually entering details such as Availability Zones or AMI IDs, Terraform can automatically retrieve the latest information from AWS at runtime.

The lab demonstrates how to:

Create and use Terraform data sources.

Dynamically fetch AWS Availability Zones.

Select a specific Availability Zone from the available list.

Retrieve the latest Ubuntu AMI automatically.

Deploy infrastructure using dynamically fetched values instead of fixed values.

Understand how AWS providers, AMIs, and filters work together.

In simple terms, this lab shows how Terraform can "ask AWS for information" before creating resources, making infrastructure code more flexible, reusable, and easier to maintain.

- Create a new folder named Demo Data Source.
- `createinstance.tf`
- `provider.tf`
- `variables.tf`
- Initialize Terraform
- Command:
- `terraform init`
- Verify Current Plan
- Command:
- `terraform plan`
- observe that the Availability Zone is not automatically selected.
- Run Terraform Plan Again
- Command:
- `terraform plan`
- Verify that Terraform now displays a valid Availability Zone.
- Test Different Availability Zones
- Change the index value and run the plan again.
- Example:
- `availability_zone = data.aws_availability_zones.available.names[0]`
- Command:

- terraform plan
- Understand Invalid Index Errors
- If an unavailable index is used:
- `availability_zone = data.aws_availability_zones.available.names[7]`
- Terraform returns an error because that Availability Zone does not exist.
- Add an AMI Data Source
- Create a data source to retrieve the latest Ubuntu AMI.
- Code:

```
data "aws_ami" "ubuntu" {  
    most_recent = true  
  
    owners = [  
        "099720109477"  
    ]  
  
    filter {  
        name = "name"  
  
        values = [  
            "ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"  
        ]  
    }  
  
    filter {  
        name = "virtualization-type"  
  
        values = [  
            "hvm"  
        ]  
    }  
}
```

- Use the Dynamic AMI
- Replace the hardcoded AMI value.
- Code:

```
ami = data.aws_ami.ubuntu.id
```

- Validate the Configuration
- Run Terraform Plan to verify that the latest Ubuntu AMI is selected automatically.
- Command:

```
terraform plan
```

- Test with Different AWS Regions
- Provide a region at runtime and let Terraform fetch the correct AMI automatically.
- Command:

```
terraform plan -var="region=us-east-1"
```

- Observe Dynamic Results
- Terraform automatically retrieves:
- Latest Ubuntu AMI
- Correct Availability Zone
- without requiring hardcoded values.

Part II

Lab of the Summary

In this lab, we learn how to create an AWS Security Group dynamically using Terraform Data Sources. Instead of manually entering IP addresses, Terraform automatically fetches AWS EC2 public IP ranges for selected AWS regions (US-East-1 and US-East-2) and uses them to create security group rules.

The lab demonstrates how Terraform can retrieve live AWS information, create infrastructure based on that information, launch an EC2 instance, attach a custom security group, verify everything in the AWS Console, and finally destroy all resources when finished.

This approach reduces manual work, keeps configurations up to date, and shows the power of Terraform Data Sources for building dynamic cloud infrastructure.

What We Are Doing in This Lab

- Using Terraform Data Sources to fetch AWS EC2 IP ranges.
- Filtering IP ranges for specific regions (US-East-1 and US-East-2).
- Creating a Security Group dynamically using those IP ranges.
- Allowing HTTPS (Port 443) traffic only from the fetched AWS IP ranges.
- Launching an EC2 instance.
- Verifying the Security Group and EC2 instance in AWS Console.
- Destroying all created resources after testing.
- Open Existing Terraform Project
- Navigate to the Terraform project that already contains:
 - provider.tf
 - variables.tf
 - create-instance.tf
- Remove the old security group variable if present.
- Create Security Group Configuration File
- Create a new file:
 - security-group.tf
- This file will contain the Security Group configuration.
- Create AWS IP Range Data Source
- Fetch AWS EC2 IP ranges for:
 - us-east-1
 - us-east-2
- Service:
- EC2
- Terraform automatically downloads the current AWS IP ranges.
- Create Security Group
- Create a custom Security Group.
- Configure:

- Protocol: TCP
- Port: 443 (HTTPS)
- Source: AWS EC2 IP ranges from the data source
- This means only AWS EC2 instances from those regions can access the resource on HTTPS.
- Add Tags
- Add useful tags such as:
- Creation Date
- Sync Token
- These values come directly from the AWS IP Range Data Source.
- Validate Terraform Configuration
- Run:

terraform validate

- Checks whether the Terraform syntax is correct.
- Generate Execution Plan
- Run:

terraform plan

- Terraform shows:
- EC2 Instance to be created
- Security Group to be created
- Security Group Rules
- AWS IP ranges being added
- No resources are created yet.
- Create Infrastructure
- Run:

terraform apply

- Type:

yes

- Terraform creates:
- EC2 Instance
- Custom Security Group
- Resources are deployed in AWS.
- Verify in AWS Console
- Check:
- EC2 Dashboard
- Verify:
- Instance is running
- Security Groups

- Verify:
- Custom Security Group exists
- HTTPS rule (443) exists
- AWS CIDR blocks are added automatically
- Tags are present
- Everything should match the Terraform configuration.
- Destroy Resources
- Run:

terraform destroy

- Type:

yes

- Terraform removes:
- EC2 Instance
- Security Group
- AWS account is cleaned up after testing.